

SYSTEM ARCHITECTURE

AI Creative Copy Agent

A six-stage agent pipeline that turns a raw campaign brief into channel-ready Email, Mobile and SMS copy. Every external dependency — the language model, web search, email delivery — sits behind an interface chosen by configuration, so the whole system runs with no credentials at all.

FastAPI · SQLAlchemy · Alembic React 18 · TypeScript · MUI 11 tables

177 tests

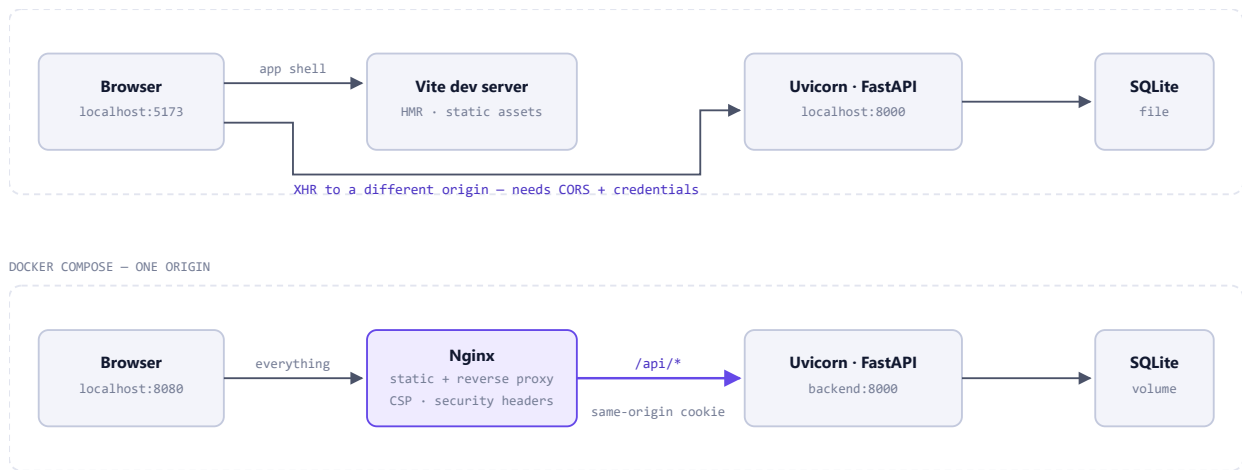
CONTENTS

- | | |
|---------------------|----------------------|
| Deployment topology | Generation lifecycle |
| The six agents | Provider seams |
| Backend layering | Authentication |
| Data model | Known constraints |

REQUEST PATH

Deployment topology

The two deployments differ by one hop, and that hop decides how the refresh cookie behaves. Under Docker, Nginx proxies `/api` to the backend, so the browser sees a single origin and the cookie is first-party. In local development the two servers are separate origins held together by CORS.

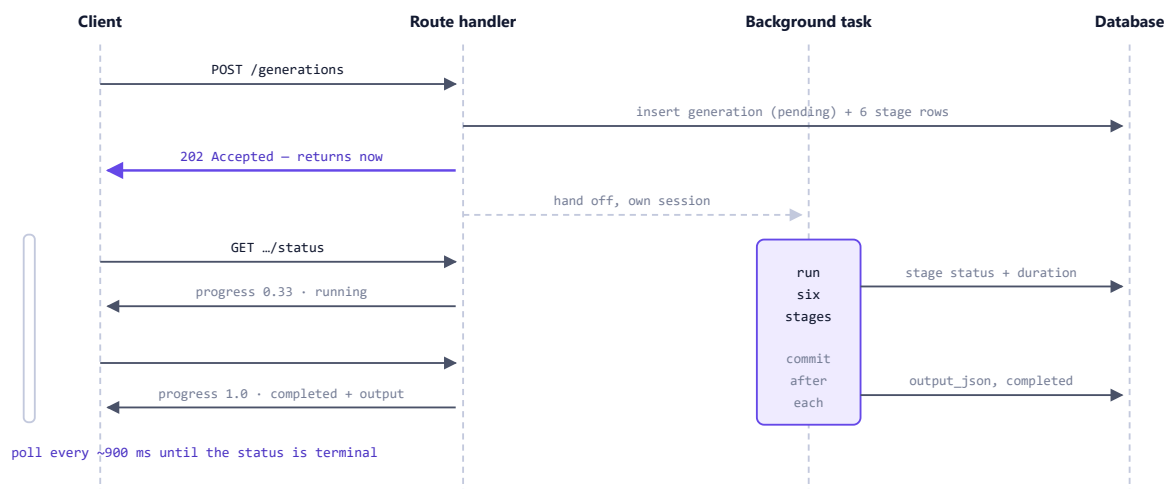


The Nginx hop is the whole difference. It removes the cross-origin XHR, which is why the production build needs no CORS allowance and why `SameSite` on the refresh cookie stops being a consideration.

ASYNC EXECUTION

Generation lifecycle

A generation takes seconds, so the request that starts one does not wait for it. The POST writes the row plus six pending stage rows and returns `202` immediately; the pipeline runs afterwards on its own database session while the client polls.

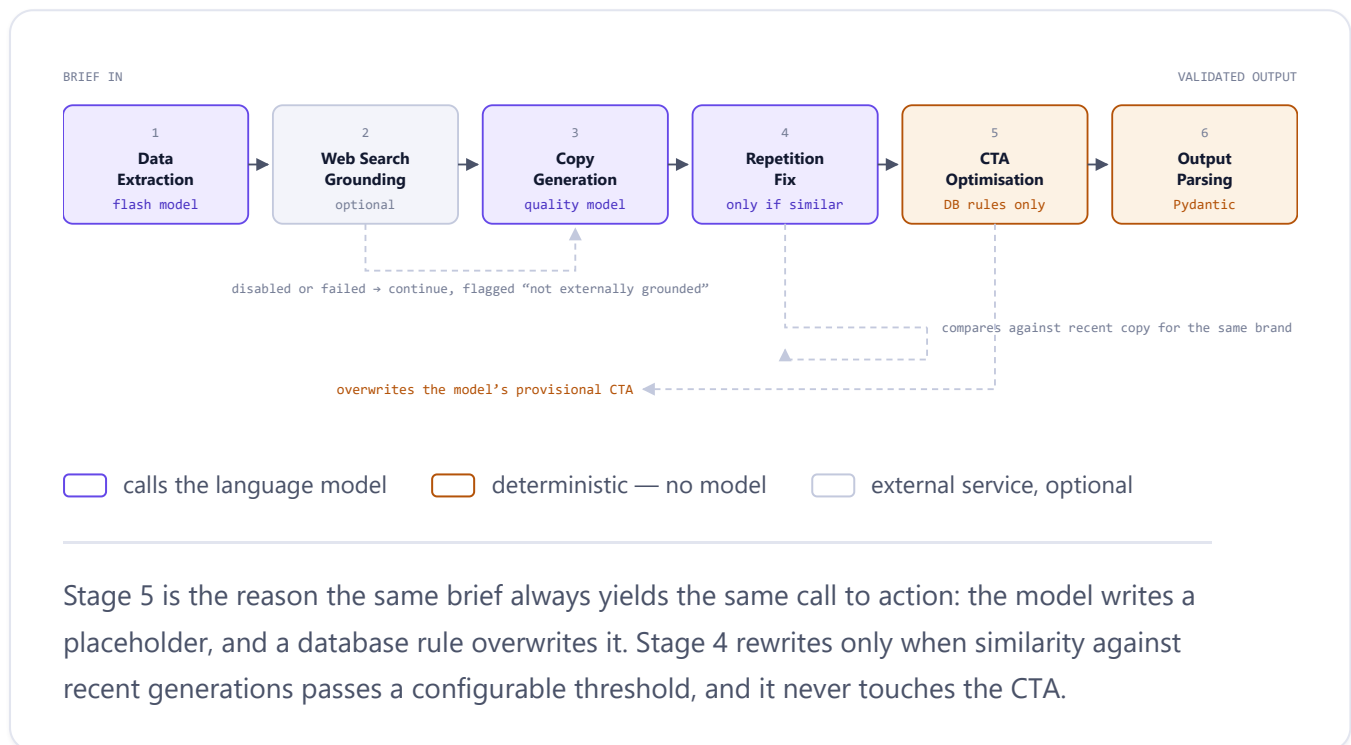


Stage rows are committed as each agent starts and finishes, which is what lets the stepper show live progress without streaming. The trade-off is stated plainly in the constraints: a restart mid-run strands that generation in `running`.

DOMAIN CORE

The six agents

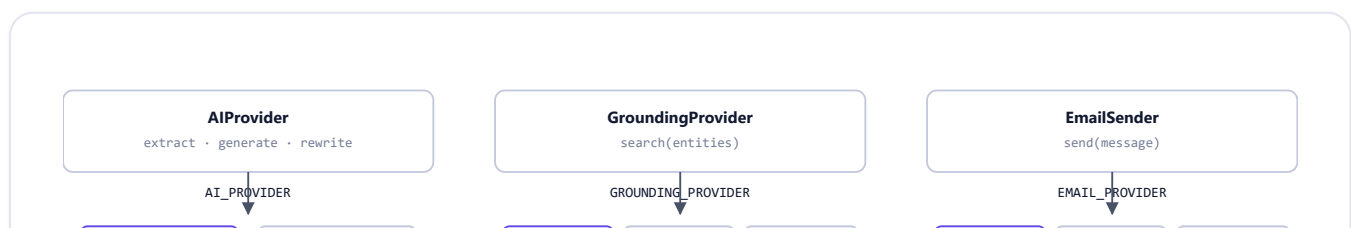
Agents read a plain `WorkflowContext` and report through a `WorkflowRecorder`. None of them touch the ORM. Two of the six never call a model at all — a distinction worth seeing, because it is what makes the call to action reproducible.



EXTENSION POINTS

Provider seams

Three interfaces isolate every external dependency. Each has a credential-free default, so a clean checkout runs the full application end to end without a single API key — and each is selected in exactly one factory function.





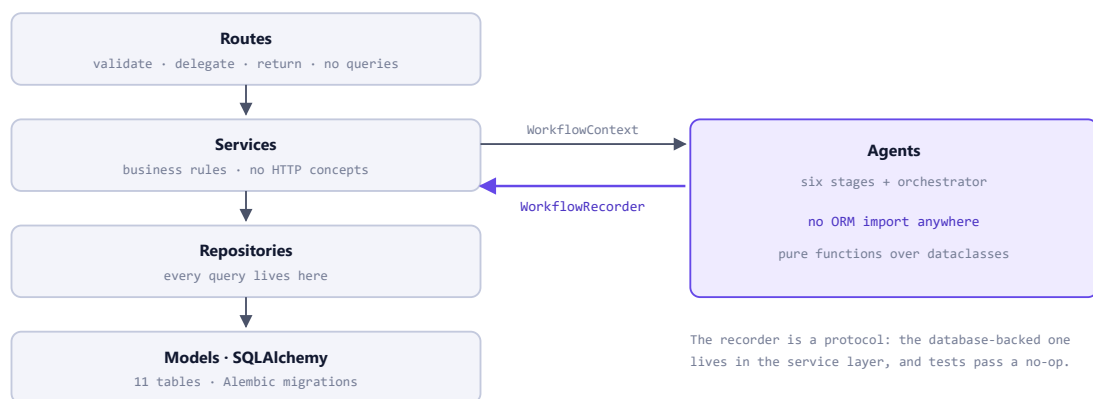
Highlighted implementations are the credential-free defaults – the application is fully usable before any key exists.

The mock AI provider is deterministic rather than random, which is what makes the repetition stage testable: the same inputs produce the same copy, so a regenerate can be asserted to differ.

CODE STRUCTURE

Backend layering

Requests travel down a fixed path, and each layer may only speak to the one beneath it. The agents sit off to the side: they receive plain dataclasses and hand progress back through a protocol, which is what keeps the domain logic free of database concerns.



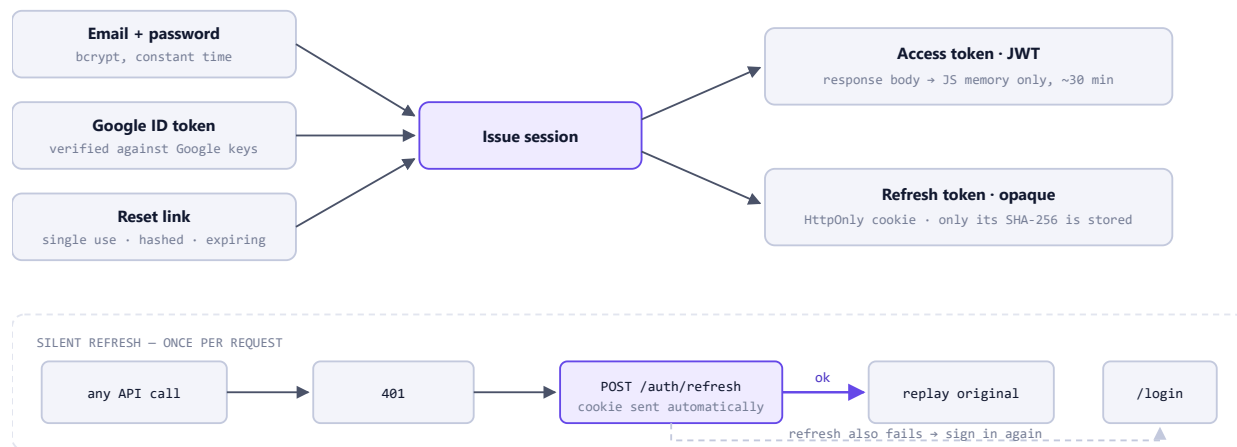
Because progress reporting is a protocol rather than a database call, the entire pipeline can be unit tested with a no-op recorder and no database at all.

SESSIONS

Authentication

The access token is short-lived and held only in JavaScript memory; the refresh

token is opaque, stored as a hash, and delivered as an `HttpOnly` cookie. Three entry paths converge on the same session issuance.

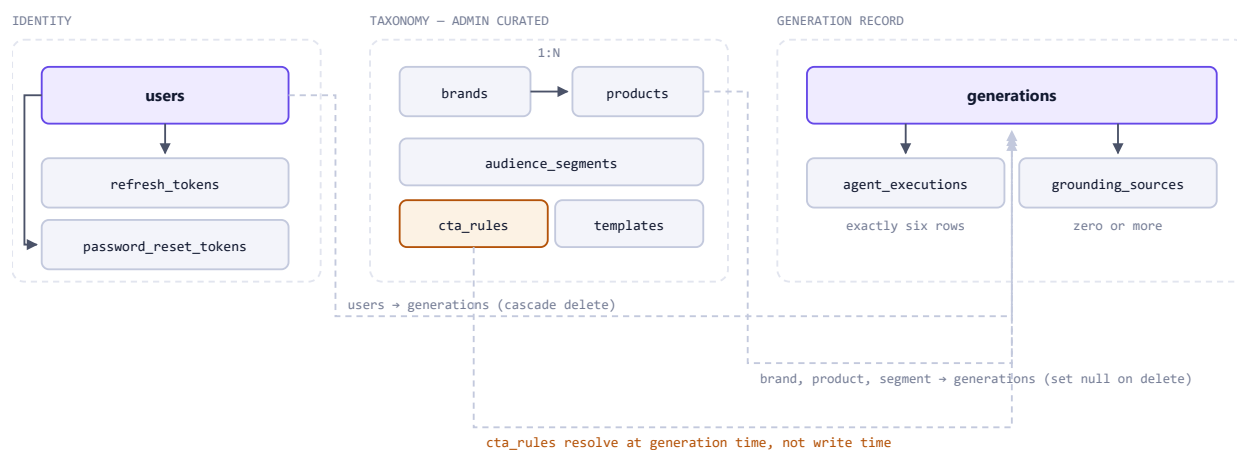


Rotation is unconditional: every refresh revokes the presented token and issues a new one. Completing a password reset revokes every outstanding session for that user.

PERSISTENCE

Data model

Eleven tables in three clusters: identity, the campaign taxonomy an administrator curates, and the generation record with its execution trail.



Taxonomy rows are never copied into a generation - deleting a brand nulls the reference and leaves the stored copy intact.

Generated copy is stored as validated JSON on the generation row so historical output

Generated copy is stored as permanent record on the generation itself, so historical copy survives any later edit or deletion of the brand, product or segment that produced it.

HONEST LIMITS

Known constraints

Each of these is a deliberate trade-off for a single-node deployment, and each has a known fix.

In-process background work

Generations run via FastAPI background tasks. A restart mid-run strands that row in `running`. A durable queue and a reaper are the production answer.

In-process rate limiting

The fixed-window counter lives in one worker's memory. More than one process means the limit multiplies; a shared store fixes it.

Polling, not streaming

Progress has a latency floor of roughly one poll interval. Server-sent events would remove it without changing the pipeline.

SQLite by default

Fine for one node. The models and migrations are portable, so the switch is a `DATABASE_URL` change plus a migration run.

VERIFIED	STATUS	NOTES
Backend suite	117 passing	Temporary SQLite, mock provider, no network
Frontend suite	60 passing	Vitest + Testing Library, API modules mocked
Gemini path	Exercised live	Real copy generated end to end
Google sign-in	Stubbed only	Verification logic tested; browser handshake unexercised
Docker images	Never built	Docker is not installed on the build machine

